

Аудит проекта и план реализации

Проект: хардкорная 3D action-RPG с вертикальным мувментом, стелсом и направленным ближним боем.

Технология: Godot 4.3 / GDScript.

Дата аудита: 22 августа 2026 г.

Автор: Manus AI.

Статус проверки. Это статический технический аудит. В комплекте присутствуют ТЗ, 10 скриптов и четыре концепт-изображения, но нет запускаемого проекта: `project.godot` , сцен `.tscn` , папки `.godot` , импортов, 3D-моделей, ригов, аудио и файлов анимаций. Поэтому ниже оценены реальные намерения ТЗ и фактическая логика в коде, но не подтверждён запуск, производительность или корректность дерева, сохранённого в редакторе.

Краткий вывод

Проект находится на стадии **раннего технического прототипа**, а не MVP и не вертикального среза. Уже есть хорошая основа для исследования идеи: движение от третьего лица, заготовка стелса в кустах, прыжок на врага, направленный замах мышью, короткий блок, здоровье, простые стаки, один прототип врага и временный UI. Однако ни одна из ключевых игровых систем ещё не собрана в устойчивую, проверяемую игровую петлю. Главный текущий блокер — не только `AnimationTree` , но и отсутствие исходной сцены, ригованных моделей и реального набора клипов, на которые дерево должно ссылаться.

По прозрачной взвешенной оценке доступных материалов готовность составляет **около 21% от пилотной версии**, но с интервалом неопределённости ± 7 пунктов из-за отсутствия запускаемого проекта. Корректнее называть это **20–30% прототипа механик** и **5–10% финального пилота**. Визуальные референсы сильные как художественная цель, но они не являются производственными 3D-ассетами.

Область	Вес	Что уже подтверждено материалами	Оценка готовности	Вывод
Базовое перемещение и камера	12%	<code>CharacterBody3D</code> , ускорение, гравитация, orbit-камера, зум	55%	Рабочая гипотеза есть; нужны сцена, Input Map, коллизии и тестирование.

Направленный бой	20%	Замах ЛКМ, выбор направления мышью, временные хитбоксы, блок, микро-стан	40%	Это прототип, не синхронизированный с анимациями и не покрывающий ТЗ целиком.
Умения и усиление	12%	Стаки 3+1, выдача сигнала готовности	10%	Нет ввода, эффектов, кулдаунов, способности 1/2/3 и расходования в игровом цикле.
ИИ и враги	12%	Один chase/attack-прототип, стелс-проверка, здоровье	15%	Нет навигации, архетипов, стай, босса и поведения по ТЗ.
Анимационная система	16%	Есть заготовка AnimationComponent.gd и сигналы	10%	Схема слишком сложна для текущей стадии, дерево и клипы не переданы.
Локация и вертикальный геймплей	14%	Только концепт и плоская тестовая сцена в описании	5%	1×1 км, маршруты, ловушки и вертикальность ещё не доказаны реализацией.
Графика, звук, VFX	6%	Арт-направление и debug-визуализация	5%	Не переданы production-ассеты; debug-кубы не заменяют визуальный слой.
UI и обратная связь	4%	НР и стаки рисуются кодом	25%	Для прототипа достаточно, для пилота нужен читаемый UX.

Сборка, тесты и production-процесс	4%	Не подтверждено	0%	Нет структуры проекта, уровней, экспортов, логов ошибок или тестовых сценариев.
Итого	100%		21,2%	Ранний технический прототип.

Что уже сделано хорошо

Код уже демонстрирует правильное стремление к компонентной модели. `Player3d.gd` делегирует здоровье, состояние, бой и стаки отдельным узлам; `StateComponent.gd` отделяет движение, прыжок, стелс и рывки; `CombatComponent.gd` выдаёт сигналы для дальнейшей привязки к визуалу. Это существенно лучше монолитного контроллера и создаёт удобную основу для доработки.

Главная механика охоты также частично выражена в коде. Игрок получает stealth при пересечении `Area3D` куста, враг не преследует игрока в стелсе, а `StateComponent` пытается запустить прыжок в сторону ближайшего врага. Удары, блокирование соответствующего направления и малое отбрасывание уже существуют как черновые интеракции. Это означает, что проект имеет **доказательство осуществимости** ключевого боевого ощущения, хотя пока не имеет готового игрового опыта.

Критические блокеры и риски

Приоритет	Проблема	Наблюдение	Последствие	Исправление
P0	Нет полной исходной структуры	Не переданы <code>project.godot</code> , <code>.tscn</code> , <code>.glb/.blend/.fbx</code> , клипы, <code>.import</code> , Input Map и ошибка из Output.	Невозможно воспроизвести ошибку AnimationTree, запустить игру или безопасно внести рабочий патч.	Передать zip корня проекта без <code>.godot/</code> , либо Git-репозиторий и текст первого красного сообщения Output.
P0	Нет ригованных 3D-ассетов	Переданы концепты	Дерево анимаций не	Сначала взять один

		персонажей/ врагов, но не скелет, модель и клипы.	может оживить концепт- изображение; ему нужны клипы в AnimationPlayer .	ригованный placeholder- персонаж или подготовить Rengar в Blender, затем делать дерево.
P0	Несоответствие ТЗ и состава сцены	Player3d.gd ждёт \$StacksComponent , а в дереве сцены из ТЗ этого узла нет. AnimationComponent тоже отсутствует в указанной архитектуре.	При запуске возможна ошибка получения узла и отсутствует гарантированная связь сигналов с AnimationTree.	Добавить оба компонента дочерними узлами Player либо заменить жёсткие пути на get_node_or_null() с понятной диагностикой.
P0	Слишком сложная первая схема AnimationTree	AnimationComponent.gd требует один root StateMachine и ещё два вложенных автомата строго по путям parameters/Attack/ playback и parameters/Block/ playback .	Малейшее отличие имени, типа узла или пути даёт null / ошибку; новичку трудно локализовать причину.	На первом проходе оставить только один корневой StateMachine и BlendSpace2D внутри состояний. Готовая упрощённая заготовка приложена отдельным файлом.
P1	Несовпадение направлений атаки	ТЗ требует три удара: сверху, справа, слева, а CombatComponent формирует UP , DOWN , LEFT , RIGHT ; блок также обрабатывает четыре направления.	Нельзя однозначно заказать и назвать клипы; DOWN не определён дизайном.	Утвердить словарь: UP , LEFT , RIGHT , KICK ; затем заменить DOWN на KICK либо формально добавить нижний удар в ТЗ.
P1	12 рывков описаны	Логично трактовать ТЗ	Получается не определённый	Зафиксировать 8 dodge-клипов

	нестрого и расходятся с кодом	как 8 уклонений + 4 кардинальных дэша. Но текущий double-tap передаёт общий <code>grid_dir</code> , поэтому может активировать диагональный dash, если одновременно удерживается другая клавиша.	набор анимаций; часть клипов будет отсутствовать или выбираться неверно.	и 4 dash-клипа. Для double-tap передавать направление нажатой клавиши, не общий вектор WASD.
P1	Боевые хитбоксы не синхронизированы с клипами	Хитбокс появляется сразу в <code>_perform_player_at_tack()</code> , а его время жизни задан числом; аналогично у врага.	Удар может наносить урон до/после визуального контакта, что разрушает доверие к направленному бою.	В прототипе привязать hit frame к данным атаки; затем запускать hitbox через call-method track или событие таймлайна клипа.
P1	Прыжок из куста не соответствует камере и целеуказанию ТЗ	Проверяется forward самого персонажа и ближайший враг. Нет raycast из центра камеры, проверки видимости, корректного выбора цели или UI-подсказки.	Прыжок может сработать «сквозь» препятствие либо по неочевидной цели.	Делать raycast от камеры по центральному лучу, отбирать врага в 11° cone и проверять path/obstacle перед стартом.
P1	Камера описана дважды	В ТЗ <code>ThirdPersonCamera</code> фигурирует и в <code>World</code> , и внутри <code>Player</code> . Скрипт камеры захватывает мышь и меняет	Две активные камеры или два обработчика ввода дают рывки, неверный <code>get_camera_3d()</code> и	Оставить ровно одну активную <code>Camera3D</code> — лучше дочернюю <code>PlayerRig/Pivot</code> .

		положение камеры.	непредсказуемы й поворот.	
P2	Враг является debug- прототипом	EnemyController.gd создаёт HP bar, стрелки, состояние, частицы и шум процедурно; AI — IDLE/CHASE/STOPPED/WINDUP .	Нет производительного, масштабируемого шаблона врага и ощущения из концептов.	Создать EnemyBase , затем отдельные Raptor , Gromp , KhaZix с data resources и настоящими сценами.
P2	Умения Ренгара отсутствуют как геймплей	Стаки считаются, но их расход не привязан к вводу; нет лечения за последние две секунды, болласа, усиленной атаки.	Core loop из T3 не замкнут.	Реализовать журнал входящего урона, ability FSM, кулдауны, цели, VFX и тесты после базового боя.

Техническая оценка существующего AnimationComponent

Сама идея связи через сигналы верная: CombatComponent эмитит attack_performed , block_started , damage_taken , а компонент анимации должен только выбрать клип и состояние. Проблема в том, что текущий AnimationComponent.gd предполагает скрытую структуру дерева, которую нельзя проверить без .tscn . Для его работы одновременно должны существовать точные состояния Motion , Dodge , Dash , Attack , Block , HitReact , два nested StateMachine Attack и Block , а во вложенных машинах — состояния Atk_UP , Atk_DOWN , Atk_LEFT , Atk_RIGHT и Block_* .

Такой дизайн слишком рано вводит два уровня навигации, кэш текущих состояний и неоднозначные переходы. В частности, _safe_attack_travel() не перезапускает тот же самый клип при повторной атаке того же направления: кэш намеренно блокирует повторный travel . В итоге может быть визуально «не нажимается вторая атака». Для одноразовых действий разумнее в первой версии использовать start(state, true) , который гарантированно начинает указанное состояние заново. travel() стоит оставлять для плавного возврата в locomotion, когда в графе действительно есть

переходы. Документация Godot подтверждает, что `travel()` идёт по кратчайшему пути графа, но телепортируется, если пути нет; `start()` запускает состояние и при `reset=true` начинает его с нуля. [1](#) [2](#)

Ниже приложен `AnimationComponent_Simple.gd`. Это **не замена игровым анимациям**, а безопасная стартовая архитектура. Он требует только один корневой `StateMachine` и пять `BlendSpace2D/Animation`-состояний: `Motion`, `Dodge`, `Dash`, `Attack`, `Block`, `HitReact`. После стабильной проверки можно наращивать прыжок, посадку, `leap` и способности.

Дорожная карта реализации

Проект не следует начинать с локации 1×1 км и полного зоопарка противников. Нужен короткий воспроизводимый «бой за 60 секунд»: персонаж появляется на серой тестовой арене, прячется в одном кусте, прыгает на одного врага, наносит/блокирует удары, получает стаки, побеждает и видит читаемую обратную связь. Это создаст реальный вертикальный срез и защитит от дорогостоящей переделки контента.

Этап	Результат	Ключевые работы	Критерий готовности
0. Зафиксировать фундамент	Запускаемая тестовая сцена и воспроизводимый baseline	Передать/собрать корень Godot, убрать дубликат камеры, создать Input Map, добавить недостающие компоненты, включить контроль версий.	Новый разработчик открывает проект, нажимает F6 и получает ту же сцену без красных ошибок.
1. Вертикальный срез движения	Приятное управление на серой арене	Один Player-сцена, одна камера, коллизии, slope/step параметры, куст, простой куб-враг, прыжок в воздухе и приземление.	Игрок бежит, прыгает, делает 8 dodge/4 dash без скольжения и без потери камеры.
2. Анимационный фундамент	Стабильный AnimationTree	Импорт одного ригованного персонажа, RESET, клипы in-place, простое дерево из руководства, debug-сцена, проверка переходов.	Все 12 перемещений, idle/motion, три удара, block/hit react воспроизводятся без null, T-pose и зависаний.

3. Бой 1v1	Одна завершённая дуэль	Синхронные hit frames, телеграф противника, направления блока, стан/knockback, смерть, HP/UI, игровые таймеры вместо debug-эффектов.	Игрок понимает, почему попал или ошибся; в 20 последовательных боях нет двойного урона и удара «в воздух».
4. Ренгар как персонаж	Замкнутый core loop	Стелс-целеуказание от камеры, leap attack, 3+1 стаки, усиленная атака, roar heal по журналу урона, bola slow, кулдауны.	Игрок использует все 3 умения и усиление в одной минутной сессии; каждый эффект тестируем.
5. Враги	Три разных игровых роли	Raptor и стая, Gromp с тяжёлым ударом/призывом, Kha' Zix как охотник/босс, data-driven параметры, NavMesh.	Каждый враг требует иной тактики; Kha' Zix появляется и завершает охоту согласно сценарию.
6. Микро-локация	Реальная вертикальная зона, не масштаб 1×1 км	150×150 м руин, озеро, тропа, две вертикальные петли, ловушка, два секрета, точки спавна.	10-минутная сессия содержит скрытность, один рискованный вертикальный маршрут и бой.
7. Полировка и расширение	Пилотная сборка	Художественные ассеты, звук, VFX, оптимизация, настройки, сохранение/перезапуск, QA, экспорт.	Внешний тестер проходит сценарий без помощи и описывает понятную боевую петлю.

Рекомендация по объёму. Локацию 1×1 км нужно делать только после микро-локации. Если начать с километра, AI, навигация, освещение, коллизии и заполнение мира поглотят ресурс до проверки того, является ли сам бой интересным.

Подробный гайд для новичка: AnimationTree в Godot 4.3

1. Простая ментальная модель

В Godot `AnimationPlayer` — это **библиотека клипов**: здесь создаются, импортируются и редактируются `Idle`, `Walk`, `Attack_Left` и остальные ролики. `AnimationTree` **не хранит анимации внутри себя**: он берёт клипы из привязанного `AnimationPlayer` и решает, какой клип воспроизводить и как смешивать его с другим. Во время игры не надо одновременно управлять и `AnimationPlayer`, и `AnimationTree`; для проигрывания и переходов следует использовать дерево, а `AnimationPlayer` оставить редактором библиотеки. 1 3

`AnimationNodeStateMachine` — это карта состояний. Например, `Motion`, `Dodge`, `Attack` — большие режимы персонажа. `BlendSpace2D` — это плоскость с координатами X/Y, на которую помещаются клипы. Если передать точку `(0, -1)`, дерево выбирает клип, расположенный «вперёд»; между точками оно смешивает соседние клипы. Godot автоматически строит треугольники смеси, если не отключать `Auto Triangles`. 1

Главное правило для первой версии: один `AnimationPlayer` + один `AnimationTree` + один корневой `StateMachine`. Не делайте для атак, блока и каждой способности отдельные вложенные автоматы, пока базовое дерево не работает.

2. Прежде чем открыть AnimationTree

Нельзя получить настоящую 3D-анимацию из прикреплённого concept art. Нужны: 3D-меш, `Skeleton3D` /риг, набор клипов и импортированная модель (чаще `.glb`). Если своего персонажа пока нет, используйте **один легальный ригованный placeholder**, чтобы закончить технический слой. Затем тот же `AnimationTree` можно привязать к финальной модели, если имена клипов и скелет совместимы.

Проверка до начала	Что должно быть	Как проверить
Модель	Открывается как сцена, видны <code>MeshInstance3D</code> и <code>Skeleton3D</code>	Двойной клик по <code>.glb</code> /сцене в <code>FileSystem</code> .
Риг	Одна логичная иерархия костей, <code>bind pose</code> без искажений	В 3D viewport включить <code>Skeleton</code> .

Клип покоя	Idle виден в выпадающем списке AnimationPlayer и зациклен	Нажать Play в AnimationPlayer.
RESET	Клип RESET на одном кадре, содержащий исходную позу/свойства	Проверить список AnimationPlayer. RESET важен для предсказуемого blending. 1
Движение	Для старта все locomotion-клипы in-place : клип не переносит root персонажа по сцене	При предпросмотре модель шагает на месте.
Сцена	У Player ровно одна активная Camera3D и один AnimationPlayer	Remote/Scene tree и Inspector.

Для этой игры на первом проходе используйте **in-place анимации**. В таком режиме StateComponent управляет CharacterBody3D.velocity и коллизиями, а клип отвечает за позу. Не включайте root motion параллельно с существующей физикой, иначе персонаж будет перемещаться дважды. Root motion — полезный следующий этап; Godot позволяет забирать его дельту через AnimationTree, но это отдельная интеграционная задача. 1

3. Минимальная сцена Player

Откройте сцену игрока и приведите её к следующему смысловому виду. Названия важны: скрипты используют их как пути.

Plain Text

```

PlayerCharacterBody3D (CharacterBody3D) [Player3d.gd, группа player]
├─ VisualModel (Node3D / инстанс модели с Skeleton3D)
├─ CollisionShape3D
├─ ThirdPersonCamera (Camera3D) [ThirdPersonCamera.gd, Current = On]
├─ HealthComponent [HealthComponent.gd]
├─ CombatComponent [CombatComponent.gd]
├─ StateComponent [StateComponent.gd]
├─ StacksComponent [StacksComponent.gd]
├─ AnimationPlayer [хранит все клипы]
├─ AnimationTree [Active = On]
└─ AnimationComponent [AnimationComponent_Simple.gd]
```

Удалите или выключите вторую Camera3D в `World`. В `AnimationTree` укажите поле **Anim Player** на соседний `AnimationPlayer`; затем включите **Active**. Если поле не назначено, дерево не знает, где искать клипы. `AnimationTree` действительно является контроллером переходов поверх `AnimationPlayer`, а не независимым хранилищем анимаций. 1 3

В компоненте `AnimationComponent` в Inspector перетащите узел `AnimationTree` в экспортируемое поле `anim_tree`. Не полагайтесь на «кажется, он рядом»: явная ссылка делает ошибку видимой и не зависит от будущего переименования.

4. Сначала заведите шесть тестовых клипов

Не пытайтесь сразу создать 30 состояний. Сначала в `AnimationPlayer` должны реально проигрываться: `RESET`, `Idle`, `Walk_F`, `Dodge_F`, `Dash_F`, `Attack_UP`, `Block_UP`, `HitReact`. Пусть даже временная модель — куб, а клипы только меняют её наклон/scale. Цель этого шага — доказать, что импорт, пути и дерево работают.

Если клипы находятся в `.glb`, лучше сначала открыть `AnimationPlayer`, по очереди выбрать каждый клип и нажать предпросмотр. Если хотя бы один не виден там, `AnimationTree` не сможет его «починить». Если клипы создаются вручную, создайте новый `Animation Library`, назовите клип точно как в таблице и добавляйте только `transform tracks` модели/костей, а не перемещение `CharacterBody3D` по миру.

5. Создайте корневое дерево без вложенных машин

В Inspector выберите `AnimationTree` → `Tree Root` → **New AnimationNodeStateMachine**, затем нажмите значок редактирования дерева. Создайте следующие состояния через правый клик в пустом пространстве:

Plain Text

```
Motion      : AnimationNodeBlendSpace2D
Dodge       : AnimationNodeBlendSpace2D
Dash        : AnimationNodeBlendSpace2D
Attack      : AnimationNodeBlendSpace2D
Block       : AnimationNodeBlendSpace2D
HitReact    : AnimationNodeAnimation
```

Установите `Motion` как Start Node (в Godot 4.3 также можно включить `Autoplay on Load`). Это важно: `StateMachine` должен быть запущен до команды `travel()`. 1 2

Создайте переходы так, чтобы любые одноразовые действия возвращались в движение. Для переходов от `Dodge`, `Dash`, `Attack`, `Block`, `HitReact` к `Motion` поставьте **Switch Mode = At End, Auto Advance = On** и небольшое `Xfade Time` 0,04–0,10 секунды.

Переходы из Motion в Dodge/Dash/Attack/Block могут быть Immediate с малым blend. У HitReact поставьте более высокий приоритет и сделайте пути из Motion/Attack/Block. В Godot At End ждёт окончания текущего состояния; Auto Advance запускает такой переход автоматически. 1

Переход	Режим	Auto Advance	Xfade	Зачем
Motion → Dodge/Dash	Immediate	Нет	0,04 с	Рывок должен быстро отвечать на ввод.
Dodge/Dash → Motion	At End	Да	0,06 с	Один ролик завершается и управление возвращается естественно.
Motion → Attack/Block	Immediate	Нет	0,05 с	Бой должен быть отзывчивым.
Attack/Block → Motion	At End	Да	0,08 с	Клип не обрывается до recovery.
Любое боевое состояние → HitReact	Immediate	Нет	0,03 с	Получение урона приоритетнее позы.
HitReact → Motion	At End	Да	0,08 с	Персонаж возвращается к нейтральному состоянию.

6. Настройте Motion BlendSpace2D

Откройте узел Motion двойным кликом. В Inspector выставьте диапазоны Min Space = (-1, -1) и Max Space = (1, 1) . Нажимайте Add Point, выбирайте AnimationNodeAnimation , а затем необходимый клип. Используйте единую систему координат:

Клипы Motion	Координата X,Y	Loop
Idle	(0, 0)	Да

Walk_F	(0, -1)	Да
Walk_B	(0, 1)	Да
Walk_L	(-1, 0)	Да
Walk_R	(1, 0)	Да
Walk_FL / FR / BL / BR, если есть	соответствующие диагонали	Да

В приложенном компоненте координата берётся из локальной скорости: `Vector2(local_velocity.x, local_velocity.z)`. В обычной ориентации 3D Godot «вперёд» — отрицательная ось Z, поэтому forward даёт (0, -1). Не меняйте знак в одних состояниях и не оставляйте старый в других: именно такая несогласованность часто создаёт «идёт назад при W».

Для временного теста можно начать с Idle + четырёх направлений. BlendSpace2D способен смешивать точки внутри автоматически сформированных треугольников; ручное рисование треугольников почти никогда не нужно на начальном этапе. 1

7. Настройте именно 12 уклонений и рывков

Зафиксируйте design-контракт: **8 уклонений + 4 дэша = 12 клипов**. Это полностью покрывает формулировку ТЗ и не раздувает объём.

Узел	Клип	Координата BlendSpace2D	Условие
Dodge	Dodge_F	(0, -1)	W + Q/E-логика, результирующее направление вперёд
Dodge	Dodge_FL	(-1, -1)	диагональ вперёд-влево
Dodge	Dodge_L	(-1, 0)	влево
Dodge	Dodge_BL	(-1, 1)	диагональ назад-влево
Dodge	Dodge_B	(0, 1)	назад
Dodge	Dodge_BR	(1, 1)	диагональ назад-вправо

Dodge	Dodge_R	(1, 0)	вправо
Dodge	Dodge_FR	(1, -1)	диагональ вперёд-вправо
Dash	Dash_F	(0, -1)	double-tap W
Dash	Dash_B	(0, 1)	double-tap S
Dash	Dash_L	(-1, 0)	double-tap A
Dash	Dash_R	(1, 0)	double-tap D

В текущем `StateComponent.gd` `dash` вызывается с `grid_dir`, собранным из всех удерживаемых клавиш. Исправьте строку вызова в ветке `double-tap` так, чтобы конкретная клавиша давала только одну из четырёх кардинальных координат:

Plain Text

```
if current_key != -1:
    var now := Time.get_ticks_msec() * 0.001
    if current_key == _last_tap_key and (now - _last_tap_time) < double_tap_win:
        var dash_dir := {
            KEY_W: Vector2(0, -1),
            KEY_A: Vector2(-1, 0),
            KEY_S: Vector2(0, 1),
            KEY_D: Vector2(1, 0)
        }[current_key] as Vector2
        _start_action(dash_dir, ActionType.DASH)
        _last_tap_time = 0.0
        _last_tap_key = -1
    return
```

Так дерево и код используют одну спецификацию: у Dash всего четыре точки, а не непредсказуемые диагонали.

8. Настройте атаки и блок без вложенного автомата

Узлы `Attack` и `Block` также являются `BlendSpace2D`. Для первого рабочего набора оставьте три направления из ТЗ. Поле `DOWN` не добавляйте, пока оно не утверждено как «нижний удар» или не заменено на `kick`.

Узел	Клип	Координата
Attack	Attack_UP	(0, -1)

Attack	Attack_LEFT	(-1, 0)
Attack	Attack_RIGHT	(1, 0)
Block	Block_UP	(0, -1)
Block	Block_LEFT	(-1, 0)
Block	Block_RIGHT	(1, 0)
HitReact	HitReact_Front	не нужен; это одиночный AnimationNodeAnimation

После утверждения kick добавьте `Kick` как отдельное состояние или четвёртую точку `Attack`. Более чистый вариант — отдельное `Kick`-состояние, поскольку у него иная дистанция, hitbox и правило пробивания блока.

Скопируйте приложенный `AnimationComponent_Simple.gd` в папку `scripts/`, прикрепите его к дочернему узлу `AnimationComponent` и в Inspector назначьте `PlayerAnimTree`. Скрипт ожидает точные имена состояний и путей:

Plain Text

```
parameters/playback
parameters/Motion/blend_position
parameters/Dodge/blend_position
parameters/Dash/blend_position
parameters/Attack/blend_position
parameters/Block/blend_position
```

Это не «магические» строки: это пути свойств, которые Godot показывает в разделе **Parameters** выбранного `AnimationTree`. При наведении на параметр редактор показывает его точный путь; копируйте его, а не перепечатывайте. Документация прямо рекомендует управлять параметрами дерева через эти `property paths`. ¹

9. Подключите прыжок и leap только после базового дерева

Когда первые состояния безошибочны, добавьте `JumpStart`, `Air`, `Land`, `Leap` и `Death`. Каждый из них сначала должен быть обычным `AnimationNodeAnimation`, не `BlendSpace`. Добавьте переходы:

Plain Text

Motion → JumpStart → Air → Land → Motion
Motion → Leap → Land → Motion
Any state → Death

Для нормального прыжка лучше ввести ясные сигналы `jump_started` , `left_floor` , `landed` в `StateComponent` , а не угадывать момент по случайному кадру. Для `Leap` уже есть `state_changed("leap", true/false)` , но нужна отдельная проверка, что атака в полёте не вытесняет клип прыжка некорректно. На первом этапе выберите правило: leap-атака играет Attack-клип с отдельным флагом, но физическое перемещение продолжает контролировать `StateComponent` .

В ТЗ задан cone 11°. Текущее значение `FACING_THRESHOLD = 0.98` соответствует приблизительно 11,5° полуугла, но считается относительно тела игрока и ближайшего врага, не центра камеры. Для дизайнерски точного leap в будущем используйте camera-centered RayCast3D + проверку линии видимости, а не только `get_nodes_in_group("enemy")` .

10. Пять обязательных проверок после каждого изменения

Тест	Что сделать	Ожидаемый результат
1. Клипы	Запустить каждый клип кнопкой Play в AnimationPlayer	Нет T-pose, неверных костей или перемещения тела по миру.
2. Дерево	Включить AnimationTree, выбрать Motion, двигать <code>blend_position</code> в Inspector	Idle и Walk плавно меняются без кода.
3. Сигнал	Временно вызвать <code>_restart_root_state(&"Attack")</code>	Attack запускается с первого кадра каждый раз.
4. Возврат	Дождаться конца dodge/attack	Автоматический переход возвращает в Motion, нет застревания.
5. Игра	Протестировать все 12 направлений 10 раз подряд	Нет отсутствующего клипа, диагонального dash, скольжения или ошибки Output.

Откройте панели **Debugger → Errors, Debugger → Stack Trace, Remote** и Parameters у AnimationTree во время запущенной игры. Новичку полезно сначала смотреть не на модель, а на два значения: текущий state playback и `blend_position`. Если они меняются правильно, а модель — нет, проблема в клипах/риге. Если они не меняются, проблема в событиях/пути параметра.

11. Таблица типичных ошибок AnimationTree

Симптом	Вероятная причина	Что сделать буквально
<code>AnimationTree not assigned</code>	В export-поле <code>anim_tree</code> не указан узел	Выбрать AnimationComponent → Inspector → перетащить AnimationTree в поле.
<code>Invalid get index 'playback'</code> или <code>root_playback == null</code>	Корень дерева не StateMachine или путь написан неверно	Установить Tree Root = AnimationNodeStateMachine ; проверить <code>parameters/playback</code> .
<code>parameters/Attack/playback</code> не существует	Attack — BlendSpace, но старый скрипт ждёт nested StateMachine	Использовать приложенный упрощённый скрипт либо построить вложенный автомат в точности как ожидает старый. Не смешивать схемы.
Ошибка про неизвестное состояние	В коде и StateMachine разные имена/регистр	Сверить <code>Attack</code> , <code>Block</code> , <code>Motion</code> , <code>HitReact</code> посимвольно. <code>attack</code> и <code>Attack</code> — разные имена.
Анимация проигрывается в редакторе, но не в игре	Tree не Active, Anim Player не назначен либо другой скрипт вызывает AnimationPlayer напрямую	Включить Active; назначить Anim Player; во время игры управлять только AnimationTree. 3
После первой атаки вторая того же направления не играет	Логика не перезапускает уже выбранный под-state	Для одноразового действия вызывать <code>root_playback.start(state, true)</code> или использовать OneShot. 2
W воспроизводит бег назад	Разные соглашения по оси Z между кодом и BlendSpace	Зафиксировать координату forward (0,-1) во всех spaces; проверить local velocity.

Модель распадается/дёргается при смешивании	Нет корректного <code>RESET</code> , несовместимые скелеты или ключи	Добавить базовую позу в <code>RESET</code> , убедиться в одной skeleton hierarchy и проверить clip tracks. 1
Персонаж скользит или проходит двойную дистанцию	Root motion клипа включён вместе с <code>CharacterBody3D.velocity</code>	На прототипе сделать клипы in-place. Переход к root motion выполнять отдельной задачей. 1
<code>travel()</code> как будто игнорирует путь	Нет перехода в графе или машина не запущена	Задать Start Node/Autoplay или вызвать <code>start</code> ; создать переходы. При отсутствии пути Godot телепортирует состояние. 1 2

Практический маршрут самостоятельной доработки

Первый рабочий вечер нужно посвятить только проверке инфраструктуры: открыть проект, убрать вторую камеру, добавить `StacksComponent` и `AnimationComponent`, проверить, что Player запускается без красных ошибок, и сделать `Idle` → `Walk_F` в дереве. На второй вечер собрать Motion BlendSpace и 12 dodge/dash с любыми placeholder-клипами. Третий — Attack/Block/HitReact и корректный автоматический возврат. Только после этого связывать ударные кадры с хитбоксами и добавлять jump/leap.

Не пытайтесь одновременно «починить дерево», добавить новые модели, переделать боевую механику и построить джунгли. В этом проекте AnimationTree должен стать **пассивным визуальным потребителем состояний**: `StateComponent` решает движение, `CombatComponent` решает попадания и блоки, `AnimationComponent` читает их сигналы и выбирает позу. Это разделение значительно проще отлаживать и расширять.

Что нужно передать для самостоятельного завершения проекта

Чтобы я мог не просто подготовить схему, а собрать и протестировать рабочую версию, нужен единый архив корня проекта или репозиторий со следующим составом.

Обязательно	Для чего
-------------	----------

project.godot , все .tscn , папка scripts/ , assets/	Открыть и запускать проект в той же конфигурации.
Все .glb , .blend , .fbx , textures и аудио, которые реально подключены к сцене	Увидеть skeleton, AnimationPlayer и фактические имена клипов.
Сцена Player и текущая сцена World	Исправить NodePath, камеру, коллизии, группы, AnimationTree и экспорты.
Скрин Debugger/Output с первой красной ошибкой и версия Godot	Быстро отделить ошибку дерева от ошибки импорта/скрипта.
Управляющие решения	Точно утвердить 3 или 4 направления атаки; какой input для kick; 8+4 ли это для 12 movement-клипов; использовать ли готовые placeholder-модели.

Не включайте в архив .godot/ и экспортированные сборки, если архив получается слишком большим; они пересоздаются. Но **не удаляйте файлы моделей, сцен и исходных клипов**. После получения полного проекта правильный порядок работ: сначала запуск и устранение ошибок, затем один работающий Player-срез, затем анимации, после чего бой/умения и контент.

Ссылки

[1] Godot 4.3 — Using AnimationTree

[2] Godot 4.3 — AnimationNodeStateMachinePlayback

[3] Godot 4.3 — AnimationTree